

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Чувашский государственный университет имени И.Н. Ульянова»

Факультет информатики и вычислительной техники

Кафедра вычислительной техники

ТЕОРИЯ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ И МЕТОДЫ ТРАНСЛЯЦИИ

Расчетно-графическая работа (часть 1)

Лексический анализ

Выполнил:

студент группы
ИВТ-42-23
Иванов И.Л.

Руководитель:

доцент Павлов Л.А.

Чебоксары 2026

Оглавление

Задание к РГР	3
Введение	4
1. Описание языка	5
2. Лексические классы языка	7
3. Синтез конечных автоматов-распознавателей	8
4. Структуры данных и алгоритмы	11
5. Программная реализация лексического анализа	15
Заключение	18
Список использованной литературы	18

Задание к РГР

1. Разработать синтаксис простого учебного языка программирования, используя нотацию БНФ или РБНФ.
2. Разработать программный модуль лексического анализа (сканер).

Общие требования к учебному языку:

- язык должен быть со строгой явной статической типизацией, т.е. типы всех объектов должны быть объявлены в специальном разделе описаний и не допускается неявное преобразование типов;
- должны быть ключевые слова, обозначающие начало и конец программы;
- программа должна состоять из раздела описаний и раздела (последовательности) операторов;
- должно быть как минимум три простых предопределенных базовых типа: целый, вещественный, логический;
- для арифметических выражений должны быть определены как минимум операции сложения, вычитания, умножения, деления;
- для логических выражений должны быть определены операции дизъюнкции (ИЛИ), конъюнкции (И), отрицания (НЕ) и шесть операций отношения;
- обязательным оператором является оператор присваивания;
- текст программы должен допускать использование комментариев.
- оператор цикла с предусловием.

Введение

Цель работы – закрепление теоретических знаний, приобретение умений и практических навыков формального описания синтаксиса языка программирования и формирования перечня семантических соглашений. Изучение методов построения лексических анализаторов (сканеров), способов построения распознавателей для регулярных языков; получение практических навыков синтеза, детерминизации и минимизации распознающих конечных автоматов и их программной реализации, выбора структур данных для представления таблиц сканера; разработка лексического анализатора в соответствии с заданным вариантом.

Лексический анализ (сканирование) является первой фазой компиляции. Его основная задача состоит в предварительной обработке исходного текста программы, которая заключается в группировании символов входного потока в лексические единицы (*лексемы*). Для каждой лексемы сканер формирует выходной *токен* вида `<код_токена, атрибут>` для последующих фаз компиляции. *Код_токена* идентифицирует класс лексемы (*лексический класс*) и определяют работу синтаксического анализатора (рассматривается как терминал). Для удобства *код_токена* будем представлять абстрактным именем (или специальным обозначением), выделенным жирным шрифтом, и ссылаться на токен по его имени (обозначению). *Атрибут* токена обеспечивает доступ к дополнительной информации о лексеме, если лексическому классу соответствует множество лексем, и определяет трансляцию токена (семантический анализ и генерация промежуточного кода).

Часто фазы лексического и синтаксического анализа объединяют в один проход. В этом случае лексический анализатор является подпрограммой синтаксического анализатора. Когда синтаксическому анализатору требуется очередной токен, он вызывает лексический анализатор, который формирует очередной токен и возвращает управление синтаксическому анализатору. В данной РГР, поскольку еще не рассматриваются последующие фазы компиляции, вместо процедур синтаксически управляемой трансляции реализовано сохранение токена. В результате исходная программа, интерпретируемая лексическим анализатором как последовательность лексем, полностью преобразуется в последовательность соответствующих токенов.

Лексический анализатор выполняет также и другие функции. В частности, он удаляет из текста исходной программы комментарии и не несущие смысловой нагрузки пробелы, символы табуляции и символы перевода строки. Еще одной задачей является согласование сообщений об ошибках компиляции и текста исходной программы (указать каким-либо образом позицию ошибки и ее характер в тексте программы). Кроме того, лексический анализатор должен строить различные таблицы, необходимые как для собственно лексического анализа, так и для последующих фаз компиляции.

В процессе проектирования сканера необходимо решить следующие задачи:

- определить формальную грамматику заданного языка;
- выделить конструкции и элементы заданного языка (лексемы), обрабатываемые на этапе лексического анализа, и определить для них соответствующие регулярные грамматики и/или регулярные выражения;
- синтезировать распознающий конечный автомат по регулярной грамматике (регулярному выражению), детерминизировать, минимизировать и программно реализовывать полученный конечный автомат;
- выбрать оптимальную (с точки зрения требуемой памяти и времени доступа) организацию структур данных лексического анализатора;
- разработать структуру лексического анализатора, определить его функции и алгоритмы и программно его реализовать.

1. Описание языка

Синтаксис формального языка (назовем его языком *DemoLang*) представим в расширенной форме Бэкуса-Наура (РБНФ). Из существующих различных модификаций синтаксиса РБНФ используем следующий вариант.

Металингвистическая переменная (нетерминал) обозначается произвольной символьной строкой. Если нетерминал состоит из нескольких смысловых слов, то они записываются слитно или разделяются символом подчеркивания.

Терминальные символы изображаются цепочками символов, заключенными в одиночные (') или двойные (") кавычки. Открывающая кавычка должна быть точно такой же, как и закрывающая, и не должна встречаться внутри цепочки.

Левая и правая части правила разделяются метасимволом "=" (вместо "::=" в БНФ), альтернативные варианты разделяются метасимволом "|". Каждое правило заканчивается точкой.

Квадратные скобки "[" и "]" означают, что заключенная в них синтаксическая конструкция может отсутствовать.

Фигурные скобки "{" и "}" означают нуль или более повторений заключенной в них синтаксической конструкции.

Курсивом выделены понятия, подчеркивающие семантический смысл синтаксической конструкции.

Ниже приведено описание синтаксиса языка *DemoLang*.

1. Программа = "package8" "IvanovIL" ";" РазделОбъявлений РазделОператоров.
2. РазделОбъявлений = [РазделТипов] РазделПеременных
3. РазделТипов = "typedef8" ОпределениеТипа { ";" ОпределениеТипа } ";"
4. ОпределениеТипа = Имя_типа "<-" Тип
5. РазделПеременных = "var8" ОбъявлениеПеременной { ";" ОбъявлениеПеременной } ";"
6. ОбъявлениеПеременной = Тип Имя_переменной
7. Тип = Имя_Типа | СоставнойТип.
8. Имя = Буква { Буква | Цифра }.
9. СоставнойТип = "struct8" "{" Тип Имя_поля { ";" Тип Имя_поля } ";" "
10. РазделОператоров = "{" ПоследОператоров "}"
11. ПоследОператоров = Оператор { ";" Оператор } ";"
12. Оператор = ОператорПрисваивания | УправляющийОператор.
13. ОператорПрисваивания = Переменная "=" Выражение.
14. Переменная = Имя_Переменной | ОбращениеКЭлементуСоставногоТипа.
15. ОбращениеКЭлементуСоставногоТипа = Переменная "." Имя_поля
16. УправляющийОператор = "do8" ПоследОператоров "if8" "(" Выражение ")" "exit8" "enddo8"
17. Выражение = ПростоеВыраж [ОперОтношения ПростоеВыраж]
18. ОперОтношения = "<" | "<=" | ">" | ">=" | "==" | "!="
19. ПростоеВыраж = ["+" | "-"] Терм { "+" | "-" Терм } | Терм { "|" Терм }
20. Терм = Фактор { "*" | "/" Фактор } | Фактор { "&" Фактор }
21. Фактор = Целое ["." Целое] | "true" | "false" | Переменная | "(" Выражение ")" | "!" Фактор.
22. Целое = Цифра { Цифра }.

Определение нетерминала «Буква» здесь не приведено ввиду его очевидности – определяется выбранным алфавитом (обычно строчные и прописные буквы латинского алфавита).

Типы данных: integer, float, logical

Краткая характеристика языка и семантические соглашения:

- Язык удовлетворяет семантическим соглашениям, характерным для многих языков программирования (единственность именования различных объектов программы, необходимость описания идентификатора до его использования и т. п.).

- В ключевых словах и идентификаторах прописные и строчные буквы не различаются.

- Ключевые слова языка зарезервированы, их нельзя использовать в качестве идентификаторов.

- Отсутствуют именованные константы.

- Отсутствуют унарные операции $+$ и $-$, т. е. для представления, например, отрицательного числа -8 , надо использовать бинарную операцию $0 - 8$.

- Базовые типы: *integer*, *real*.

- Отсутствуют производные типы, т. е. есть только базовые простые типы.

- Предусмотрен только один оператор – оператор присваивания.

- Отсутствуют логические выражения.

- Комментарий представляет собой любую последовательность символов, заключенную в ограничители – фигурные скобки "{" и "}".

В языке (в соответствии с приведенной выше грамматикой) определена следующая приоритетность арифметических операций (в порядке убывания приоритетов):

1. Мультипликативные операции: $*$ | $/$

2. Аддитивные операции: $+$ | $-$

Пример программы на языке *DemoLang*:

```
package8 IvanovIL;
```

```
typedef8
```

```
    int <- integer;  
    myfloat <- float;  
    bool <- logical;
```

```
mystruct <- struct8
```

```
{  
    integer a;  
    int b;  
    float d;  
    myfloat c;  
    logical f;  
    bool e;  
};
```

```
var8
```

```
    integer a;  
    int b;  
    myfloat <*комментарий*>c;  
    float d;  
    bool e;  
    logical f;
```

```
mystruct s;
```

```

{
  a = <*комментарий*>10 + 20 * 30;
  b = 30 / 6 - 4;
  c = 10.0;
  e = true;
  f = !e | false & true;
  e = (1 > 2) & (10 <= 3) & (4 >= 5) | (1 <
<*комментарий*>2) | !(2 != 3);

  <*
  комментарий
  й
  *>

  do8
    s.a = 10 + 20 * 30;
    s.b = 30 / 6 - 4;
    s.c = 10.0;
    s.e = true;
    s.f = !e | <*комментарий*>false & true;

    if8 ( s<*комментарий*>.e == false)
      exit8

  enddo8;
}

```

2. Лексические классы языка

Перечень лексических классов (токенов) для языка *DemoLang* представлен в табл. 1 (в качестве формальных шаблонов токенов используются регулярные выражения).

Таблица 1. Таблица лексических классов

№	Токен	Имя	Код токена	Формальный шаблон	Значение атрибута
1	package8	pckg	lcPckg	package8	0
2	{	{	lcStart	{	0
3	}	}	lcStop	}	0
4	var8	var	lcVar	var8	0
5	do8	do	lcDo	do8	0
6	if8	if	lcIf	if8	0
7	exit8	exit	lcExit	exit8	0
8	enddo8	enddo	lcEnddo	enddo8	0
9	typedef8	typedef	lcTypedef	typedef8	0
10	struct8	struct	lcStruct	struct8	0
11	точка с запятой	;	lcSemi	;	0
12	точка	.	lcDot	.	0
13	откр. скобка	(	lcOpPar	(	0
14	закр. скобка	)	lcClPar	)	0
15	присваивание	ass	lcAss	=	0
16	присваивание типа	typeass	lsTypeAss	<-	0
17	операция сравнения	comp	lcComp	< <= > >= == !=	opL (<) opLe (<=) opG (>) opGe (>=) opE (==) opNe (!=)
18	логичИ	and	lcAnd	&	0
19	логичИЛИ	or	lcOr		0
20	логичНЕ	not	lcNot	!	0
21	логичИСТИНА	true	lcTrue	true	0
22	логичЛОЖЬ	false	lcFalse	false	0
23	аддитивная операция	+	lcAdd	+ -	opAdd (+) opSub (-)
24	мультипл. операция	*	lcMult	* /	opMult (*) opDiv (/)
25	идентификатор	id	lcId	l (l d)* l – буква, d – цифра	указатель на запись в таблице символов
26	число	num	lcNum	d+ d+.d+ d – цифра	указатель на запись в таблице символов
27	пробел	sp		(sp tab lf)+ sp – пробел, tab – табуляция, lf – перевод строки	токен не формируется

28	комментарий	com		<*X*> X – любой символ кроме *>	токен не формируется
29	маркер конца ввода	eof	lcEof		0
30	лексическая ошибка	err	lcErr		0

Для лучшей читабельности для представления кодов токенов используются имена вместо безличных числовых кодов. Поскольку лексические классы «пробел» (**sp**) и «комментарий» (**com**) не формируют токен, нет необходимости в сопоставлении им кода токена (соответствующие ячейки таблицы пусты). Первые 4 строки соответствуют ключевым словам.

Наряду с токенами, определяемыми грамматикой языка *DemoLang*, добавлены служебные токены: «маркер конца ввода» **eof** (*lcEof*) и «лексическая ошибка» **err** (*lcErr*). Для этих токенов не предусмотрены лексемы языка, поэтому соответствующие формальные шаблоны пусты.

С помощью токена **err** сканер информирует парсер о наличии лексической ошибки (парсер должен прекратить работу). Токен **eof** предназначен для информирования парсера о завершении сканирования входного текста, т. е. генерируется маркер конца ввода.

Приведенные в таблице регулярные выражения не учитывают опережающее чтение дополнительных символов для распознавания токена. Поэтому эти символы будут учтены непосредственно при синтезе конечных автоматов-распознавателей.

3. Синтез конечных автоматов-распознавателей

Прежде всего, следует решить вопрос распознавания ключевых слов. Одним из вариантов является создание специальной статической таблицы ключевых слов (тогда не надо для них строить автоматы). Для реализации бинарного поиска достаточно хранить список лексем, представляющих ключевые слова, в алфавитном порядке. При необходимости (в случае длинных ключевых слов) в структуру таблицы можно добавить поле для хранения более короткого имени (обозначения) соответствующего токена.

Для всех остальных токенов (включая и лексические классы «комментарий» и «пробел») следует построить распознающие автоматы.

Распознавателем регулярного языка является конечный детерминированный автомат $M = (K, T, \delta, k_0, F)$, где K – конечное множество состояний, T – конечный входной алфавит, $\delta: K \times T \rightarrow K$ – функция переходов, $k_0 \in K$ – начальное состояние автомата, $F \subseteq K$ – множество конечных состояний.

При синтезе автоматов будет учитываться опережающее чтение дополнительных символов и возврат этих символов во входной поток. Данная информация указывается в конечных состояниях автоматов, а именно: формируемый токен с соответствующим значением атрибута и число символов, возвращаемых во входной поток. Синтез выполняется по соответствующим регулярным выражениям (см. табл. 1). Конечные состояния автоматов обозначим отрицательными числами [2].

После синтеза автоматов для отдельных токенов (подмножеств токенов) можно выполнить их объединение в один автомат. Такое объединение легко реализуется, если первые читаемые символы для всех исходных автоматов различны. Поэтому начальные состояния всех автоматов обозначаются состоянием 0, для остальных состояний

используется сквозная нумерация (отдельно для внутренних и отдельно для конечных состояний).

Графы синтезированных автоматов, распознающих токены языка *DemoLang*, представлены на рис. 1. Конечные состояния автоматов изображены в виде прямоугольников. Дуги, помеченные символом $\perp$, означают переход в соответствующее состояние при чтении любого другого входного символа. Для конечного состояния указаны формируемый токен с соответствующим значением атрибута и число символов, возвращаемых во входной поток при выполнении опережающего чтения. Поскольку ключевые слова удовлетворяют правилам образования идентификаторов, в конечном состоянии –11 автомата для токена **id** (рис. 1, d) определяется, является данный идентификатор ключевым словом (если является, формируются соответствующие токен и, если есть, значение атрибута из таблицы ключевых слов) или нет (в этом случае формируется токен **id** с соответствующим значением атрибута). Для токенов **num** и **id** значениями атрибутов устанавливаются указатели на соответствующие записи таблицы символов (обозначение *pnt*), а для токенов +, * – соответствующие коды операций (обозначение *op*).

Поскольку первые читаемые символы в начальном состоянии 0 для всех исходных автоматов различны, легко реализуется их объединение в один автомат. В этом случае автоматы, изображенные на рис. 1, можно рассматривать как соответствующие фрагменты одного автомата, реализующего лексический анализ. Этот автомат после распознавания токена в каком-либо конечном состоянии и передачи его синтаксическому анализатору прекращает работу. После запроса следующего токена автомат начинает его распознавание с начального состояния 0. Исключением являются токены «пробел» и «комментарий», которые не передаются синтаксическому анализатору. После их распознавания автомат переводится в состояние 0, т. е. начинается распознавание следующего токена с начального состояния (причем без возврата управления парсеру).

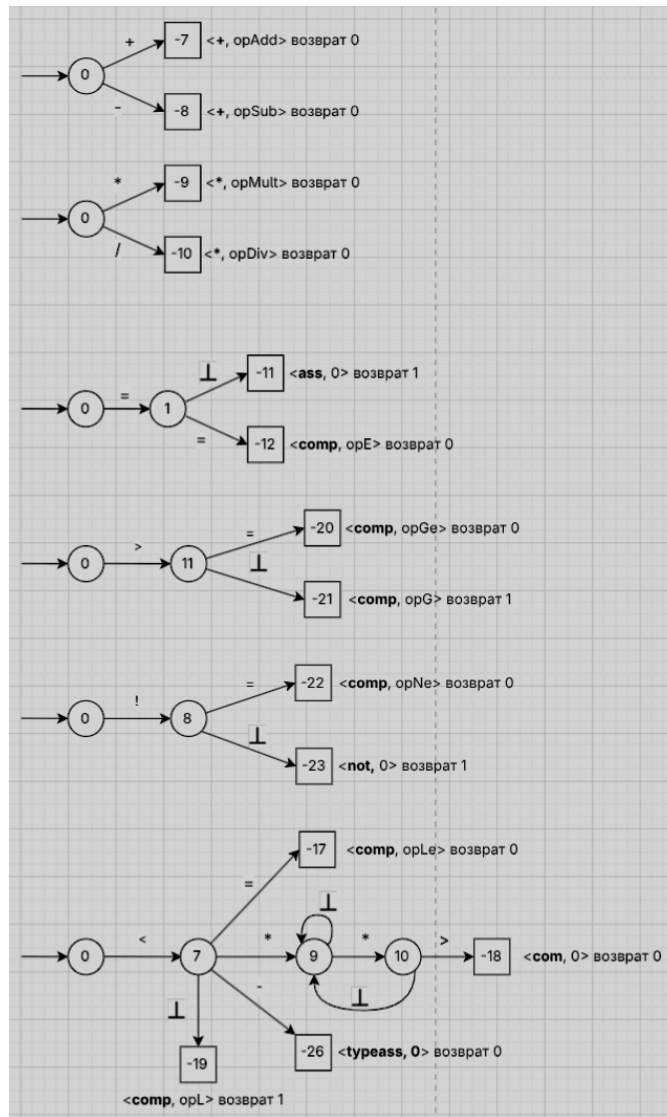
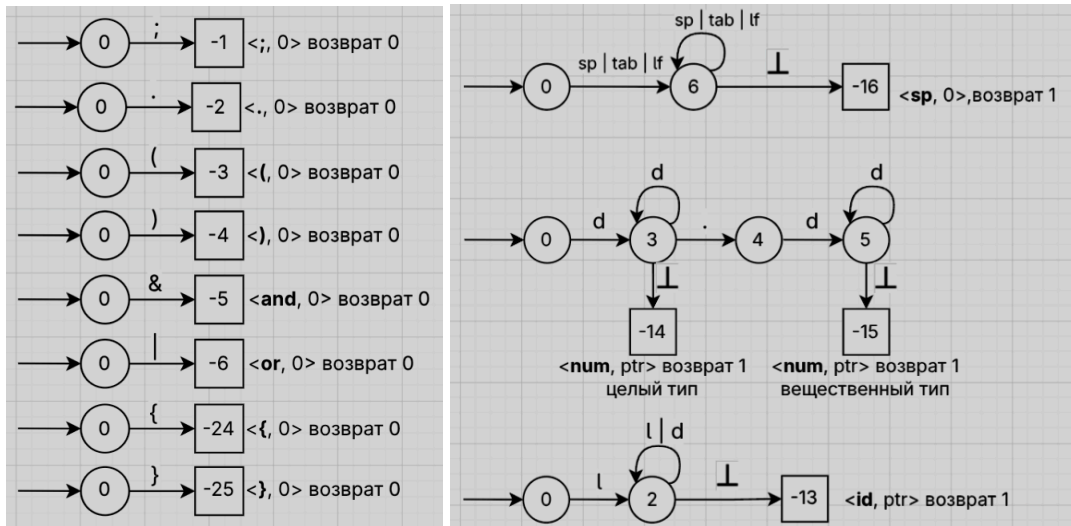


Рис. 1,2,3. Конечные автоматы-распознаватели для токенов

В общем случае после объединения автоматов в один автомат, полученный автомат может иметь неполную функцию переходов $\delta: K \times T \rightarrow K$. Все элементы множества $K \times T$, на которых функция переходов δ не определена (за исключением

конечных состояний), соответствуют лексической ошибке. Таблица переходов объединенного автомата представлена в табл. 2.

Таблица 2. Таблица переходов объединенного конечного автомата

№	Вход символ	Состояния автомата (за исключением конечных состояний)											
		0	1	2	3	4	5	6	7	8	9	10	11
1	;	-1	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
2	.	-2	-11	-13	4	err(2)	-15	-16	-19	-23	9	9	-21
3	(	-3	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
4	)	-4	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
5	+	-7	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
6	-	-8	-11	-13	-14	err(2)	-15	-16	-26	-23	9	9	-21
7	*	-9	-11	-13	-14	err(2)	-15	-16	9	-23	10	9	-21
8	/	-10	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
9	=	1	-12	-13	-14	err(2)	-15	-16	-17	-22	9	9	-20
10	<	7	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
11	>	11	-11	-13	-14	err(2)	-15	-16	-19	-23	9	-18	-21
12	!	8	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
13	&	-5	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
14		-6	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
15	l	2	-11	2	-14	err(2)	-15	-16	-19	-23	9	9	-21
16	d	3	-11	2	3	5	5	-16	-19	-23	9	9	-21
17	{	-24	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
18	}	-25	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21
19	sp	6	-11	-13	-14	err(2)	-15	6	-19	-23	9	9	-21
20	tab	6	-11	-13	-14	err(2)	-15	6	-19	-23	9	9	-21
21	lf	6	-11	-13	-14	err(2)	-15	6	-19	-23	9	9	-21
22	любой другой	err(1)	-11	-13	-14	err(2)	-15	-16	-19	-23	9	9	-21

err(1) – лексема не может начинаться с данного символа (состояние -27);

err(2) – неправильный формат числа (состояние -28);

В данной таблице строкам соответствуют входные символы, а столбцам – состояния автомата (конечные состояния не указаны, поскольку для них функция переходов не определена). В ячейке на пересечении строки и столбца записывается состояние, в которое должен перейти автомат, если в данном состоянии он считал данный входной символ. Автомат является частичным, поскольку на некоторых элементах множества $K \times T$ функция переходов не определена.

4. Структуры данных и алгоритмы

Исходный текст обрабатываемой грамматики представляет собой текстовый файл. Для облегчения реализации опережающего чтения и возврата символов во входной поток используется входной буфер, из которого лексический анализатор может выполнять чтение и в который может возвращать прочитанные символы путем простого перемещения указателя. Использование входного буфера повышает также эффективность анализатора, так как считывание блока символов обычно существенно более эффективно, чем посимвольное считывание. Используется схема буферизации, рекомендованная в [1; 7].

Буфер рассматривается как массив символов. Для распознавания лексемы требуется два указателя (индекса): f указывает на позицию первого символа лексемы, r перемещается в процессе распознавания по символам лексемы, начиная с позиции f (рис. 2). После того, как автомат, распознав лексему, переходит в конечное состояние, указатель r будет указывать на последний символ лексемы или на последний дополнительно прочитанный символ. В последнем случае дополнительно прочитанные символы возвращаются во входной поток соответствующим уменьшением значения указателя r . В итоге r будет указывать на последний символ лексемы. Для распознавания следующей лексемы указатель f устанавливается в позицию, непосредственно следующую за r .

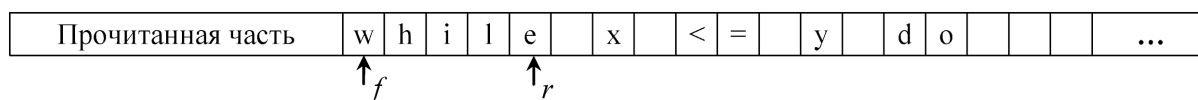


Рис. 2. Использование указателей при распознавании лексемы

Для каждой лексемы сканер формирует выходной токен вида `<код_токена, атрибут>` для последующих фаз компиляции. Код_токена идентифицирует лексический класс лексемы и определяют работу синтаксического анализатора (рассматривается как терминал). Для удобства код_токена будем представлять абстрактным именем (или специальным обозначением) и ссылаться на токен по его имени (обозначению). Атрибут токена обеспечивает доступ к дополнительной информации о лексеме, если лексическому классу соответствует множество лексем, и определяет трансляцию токена (семантический анализ и генерация промежуточного кода). Кодировка токенов и их обозначения реализуются в соответствии с табл. 1.

Выбор структуры данных для представления таблицы символов.

Для реализации лексического анализатора были спроектированы и использованы следующие структуры данных:

Перечисления для классификации токенов

- **LexCode** — перечисление (enum), описывающее все возможные типы лексем языка. Включает коды для ключевых слов (`lcPckg`, `lcVar`, `lcIf`, базовых типов данных и др.), разделителей (`lcSemi`, `lcStart`, `lcStop`), операций (`lcAss`, `lcComp`, `lcMult`) и базовых классов токенов (идентификатор `lcId`, число `lcNum`, ошибка `lcErr`, конец файла `lcEof`).
- **TokenAttribute** — перечисление атрибутов для групповых токенов. Позволяет различать конкретные операции внутри одного класса (например, для класса

`lcComp` определены атрибуты `opL`, `opLe`, `opG`, `opGe`, `opE`, `opNe`, соответствующие операциям `<`, `<=`, `>`, `>=`, `==`, `!=`).

Основные классы данных

- **Класс `Token`** — структура, описывающая сформированную лексему. Хранит три основных поля:
 - `code_` (тип `LexCode`) — код токена.
 - `attr_` (тип `int32_t`) — значение атрибута. Для операций сравнения или арифметики хранит код из `TokenAttribute`. Для идентификаторов (`lcId`) и чисел (`lcNum`) хранит **индекс (указатель)** на соответствующую запись в таблице идентификаторов.
 - `lexName_` (тип `std::string`) — строковое представление лексемы (например, имя переменной или строковое значение числа).
- **Класс `TableId_Elem`** — элемент таблицы идентификаторов. На этапе лексического анализа инкапсулирует строковое имя (`lex_`) обнаруженного идентификатора или константы. В дальнейшем структура может быть расширена полями для хранения типа данных и значения (на этапе семантического анализа).

Внутренние структуры класса `Scanner`

В классе `Scanner` используются структуры данных, обеспечивающие логику работы конечного автомата и хранение окружения:

- **Матрица переходов (`transitionTable`)** — двумерный массив `const int8_t` размерностью `[22][12]`. Строки соответствуют классам входных символов, столбцы — текущему состоянию автомата. Положительные числа в ячейках обозначают переход в следующее промежуточное состояние. Отрицательные числа являются **конечными (допускающими) состояниями**, сигнализирующими об успешном сборе лексемы или ошибке.
- **Хеш-таблица ключевых слов (`keywordMap`)** — структура `std::unordered_map<std::string, LexCode>`. Обеспечивает мгновенный поиск ($O(1)$) зарезервированных слов языка для отделения их от пользовательских идентификаторов.
- **Таблица идентификаторов (`idTable`)** — динамический массив `std::vector<TableId_Elem>`. Выполняет роль таблицы символов. Хранит уникальные имена переменных и числовые константы, встретившиеся в исходном коде.

Алгоритмы работы лексического анализатора

В основе лексического анализатора лежит алгоритм моделирования работы детерминированного конечного автомата (ДКА) с использованием явно заданной матрицы переходов. Основной метод `RunScanner(const char *text)` выполняет посимвольное чтение исходного кода и формирует поток токенов.

Основной цикл сканирования

1. **Инициализация:** Указатель чтения `i` устанавливается на начало строки. Цикл продолжается, пока не встретится символ конца строки `\0`. Для каждого нового токена текущее состояние (`currState`) обнуляется, а буфер лексемы (`lexName`) очищается.
2. **Посимвольное чтение (Работа автомата):** Внутренний цикл `while (currState >= 0)` считывает очередной символ `sym` и добавляет его в буфер `lexName`. Алгоритм вызывает метод `GetRowIndex_BySymbol(sym)` для определения класса символа (буква, цифра, разделитель и т.д.), после чего обновляет текущее состояние:
`currState = transitionTable[rowIdx][currState];`
Цикл прерывается, когда автомат переходит в отрицательное (конечное) состояние.

Обработка конечных состояний

1. **Алгоритм «Возврата» (Retract):** После выхода из промежуточного цикла сканер проверяет, требует ли конечное состояние возврата каретки (переменная `isRetract`). Это необходимо для токенов, граница которых определяется по первому символу, не входящему в их состав (например, идентификатор заканчивается пробелом или оператором). Если условие выполняется, указатель чтения `i` сдвигается на 1 назад, а последний лишний символ удаляется из строки `lexName`.
2. **Фильтрация пустых токенов:** Если автомат завершил работу в состоянии `-16` (пробельные символы) или `-18` (комментарий), сборка токена прерывается командой `continue`, так как эти элементы игнорируются транслятором.
3. **Генерация токена:** Метод `GetToken_ByState(currState)` возвращает предварительный объект `Token` на основе достигнутого отрицательного состояния.

Алгоритм разделения идентификаторов и работы с таблицами

Если предварительно собранный токен был классифицирован как идентификатор (код `lcId`), применяется алгоритм уточнения:

1. Строка `lexName` ищется в хеш-таблице `keywordMap`.
2. Если совпадение найдено, код токена `lcId` заменяется на код соответствующего ключевого слова (например, `lcPckg`, `lcInteger`, `lcDo`).
3. Если совпадение не найдено, лексема считается пользовательским идентификатором. Вызывается метод `GetTableIdElemIndex`. Если идентификатор встречается впервые, он добавляется в конец `idTable` методом `TableId_AddElem`. Позиция идентификатора в таблице сохраняется в атрибут токена (`token.SetAttr(idIndex)`).

Обработка чисел: Для числовых констант (код `lcNum`) также вызывается метод `TableId_AddElem`, который сохраняет значение числа в таблицу символов, а его индекс записывает в атрибут токена.

Завершение работы

Сформированный токен добавляется в результирующий вектор `tokens`. Если в процессе работы обнаруживается лексическая ошибка (коды состояний `-27` или `-28`), сканирование немедленно прерывается во избежание некорректного разбора, и функция возвращает список токенов, собранных до места возникновения ошибки.

5. Программная реализация лексического анализа

Выбор среды разработки

В качестве языка программирования выбран стандарт языка C++17, а в качестве среды для разработки пользовательского интерфейса — кросс-платформенный фреймворк Qt 6. Сборка проекта осуществляется с помощью системы CMake. Этот выбор сделан исходя из следующих соображений.

Прежде всего, C++ является промышленным стандартом для разработки системного программного обеспечения, включая компиляторы и трансляторы. Он обеспечивает непревзойденно высокую скорость выполнения программ и позволяет разработчику осуществлять тонкий контроль над распределением памяти.

Преимущества использования связки C++ и Qt 6:

- Кросс-платформенность: исходный код не привязан к конкретной операционной системе. Использование системы сборки CMake позволяет легко осуществлять кросс-компиляцию (например, собирать `.exe` файлы для Windows, находясь в операционной системе Linux, с использованием MinGW);
- Мощная стандартная библиотека (STL): наличие готовых высокооптимизированных структур данных, таких как динамические массивы (`std::vector`) и хеш-таблицы (`std::unordered_map`), что критически важно для эффективной реализации таблиц символов и словарей ключевых слов;
- Богатые возможности GUI: фреймворк Qt предоставляет удобные абстракции для работы с интерфейсом, включая готовые механизмы для реализации подсветки синтаксиса (`QSyntaxHighlighter`) и нумерации строк;
- Объектно-ориентированная парадигма: позволяет логически разделить архитектуру приложения на независимые классы (сканер, токен, элементы таблиц, графические виджеты).

На выбор стека технологий повлияло также желание получить глубокий практический опыт работы с современными стандартами C++ и популярным фреймворком Qt, что является высокотребовательным навыком на рынке труда.

Архитектура сканера и структуры данных

Вся логика лексического анализатора инкапсулирована в рамках объектно-ориентированной парадигмы в классе `Scanner`.

Основные методы класса:

- `std::vector<Token> RunScanner(const char *text)` — реализация сканера (цель данной части работы). Выходным параметром является динамический массив (вектор) сформированных токенов.
- `int32_t GetTableIdElemIndex(std::string lexName)` — реализует поиск пользовательского идентификатора в таблице символов.
- `int32_t TableId_AddElem(std::string lexName)` — реализует добавление новой записи в таблицу символов (идентификаторов и чисел).

Локальные для сканера структуры данных:

Для распознавания принадлежности символов к буквам или цифрам не используются жестко заданные множества. Вместо этого применяются стандартные функции стандартной библиотеки C `<cctype>`: `std::isalpha` (для букв) и `std::isdigit` (для цифр), что обеспечивает безопасную и стандартизированную обработку символов.

Таблица переходов автомата задается как константный двумерный массив:

```
const int8_t transitionTable[22][12],
```

где строки (от 0 до 21) соответствуют классам входных символов, а столбцы (от 0 до 11) — внутренним промежуточным состояниям автомата.

Для отделения зарезервированных ключевых слов (например, `package8`, `var8`, `integer`) от пользовательских идентификаторов используется хеш-таблица `std::unordered_map<std::string, LexCode> keywordMap`. Это обеспечивает алгоритмическую сложность поиска $O(1)$, в отличие от линейного перебора.

Организация интерфейса и алгоритм работы

Исходный текст тестовой программы размещается в специализированном визуальном компоненте `CodeEditor`, унаследованном от `QPlainTextEdit`. Данный компонент расширен функционалом автоматической нумерации строк и динамической подсветки синтаксиса ключевых слов синим цветом. С помощью стандартных диалоговых окон `QFileDialog` реализована возможность загружать программу из текстового файла и сохранять отредактированный текст (кнопки «Загрузить» и «Сохранить»). Перед передачей в сканер исходный текст конвертируется в массив символов `const char*`, что делает его удобным для использования в качестве буфера ввода.

Алгоритм работы автомата представляет собой цикл `while`, который посимвольно читает входной буфер, пока не встретит символ конца строки `\0`. Функция `GetRowIndex_BySymbol(sym)` возвращает номер строки в таблице переходов для текущего символа.

Специальное хранение множества конечных состояний не требуется, поскольку они закодированы отрицательными числами. Поэтому условие продолжения работы конечного автомата реализуется простым неравенством `currState >= 0`.

Если автомат переходит в отрицательное состояние, цикл прерывается. Далее алгоритм проверяет, требует ли данное конечное состояние «возврата» каретки (сдвига указателя чтения на один символ назад). Затем с помощью функции `GetToken_ByState` на основе кода конечного состояния формируется токен, который записывается в итоговый массив.

Инструкция пользователя и результаты тестирования

Разработанное кросс-платформенное приложение поставляется в виде исполняемого файла (например, `compl_scanner.exe` для ОС Windows). Приложение имеет современный графический интерфейс, разделенный на две основные рабочие области.

Левая панель включает в себя редактор исходного кода с подсветкой синтаксиса. Пользователь может ввести текст программы вручную или загрузить готовый тест из файла `.txt`. Правая панель предназначена для вывода результатов работы сканера и представляет собой таблицу (`QTableWidget`) из трех колонок: «Токен», «Атрибут» и «Лексема».

Для запуска лексического анализа служит кнопка «Сканер». При ее нажатии происходит очистка предыдущих результатов, после чего таблица мгновенно заполняется распознанными лексемами. Если сканер распознает переменную или число, в столбце «Атрибут» выводится индекс данного элемента в таблице символов. Если в процессе анализа встречается неизвестный символ (например, не предусмотренный грамматикой), сканер генерирует токен ошибки (код `err`) и корректно прерывает дальнейший разбор, избегая заикливания.

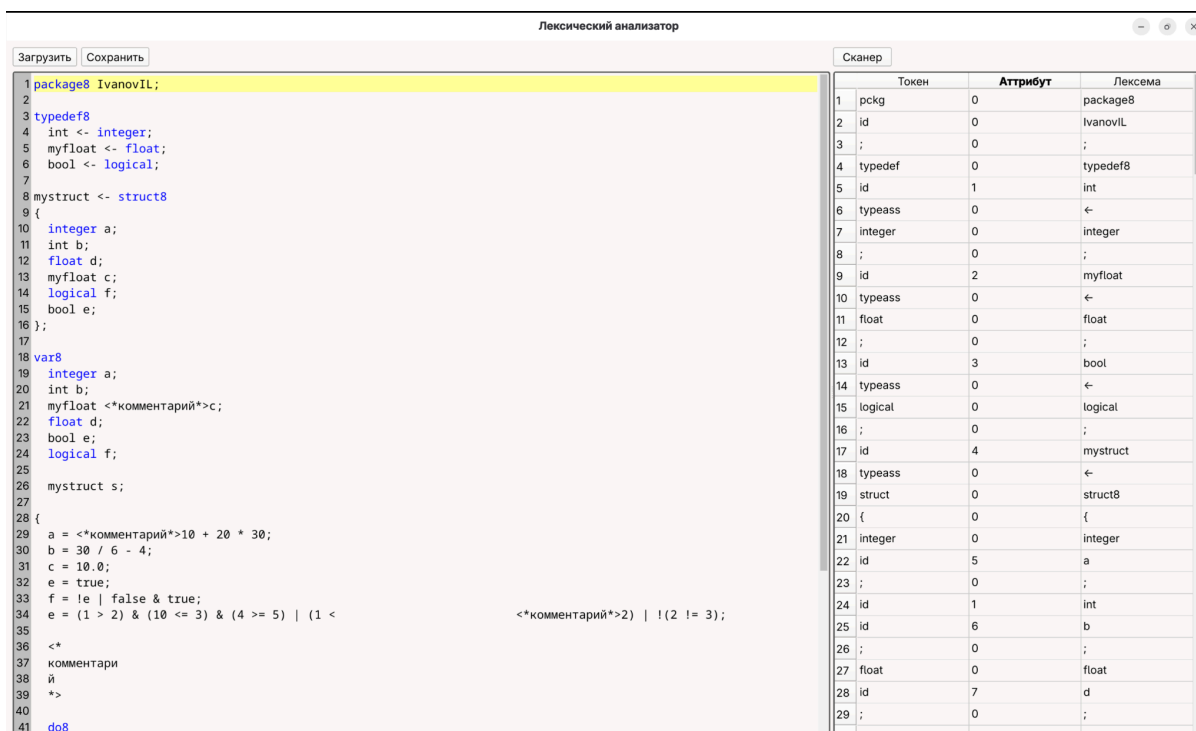


Рис. 3. Интерфейс приложения

Заключение

В процессе выполнения расчетно-графической работы

- разработан язык программирования *DemoLang*, синтаксис описан с помощью РБНФ;
- изучены методы построения лексических анализаторов;
- изучены способы построения распознавателей для регулярных языков;
- разработаны структуры данных для представления таблиц сканера;
- получены практические навыки синтеза, детерминизации и минимизации распознающих конечных автоматов и их программной реализации;
- получены практические навыки разработки языка программирования и модуля лексического анализа.

Список использованной литературы

1. Ахо, А. Компиляторы: принципы, технологии и инструментарий / А. Ахо, М. Лам, Р. Сети, Д. Ульман. – 2-е изд. – М.: Вильямс, 2008. – 1184 с.
2. Павлов, Л.А. Лексический и синтаксический анализ: учеб. пособие / Л.А. Павлов. – Чебоксары: Изд-во Чуваш. ун-та, 2022. – 152 с.
3. Павлов, Л.А. Структуры и алгоритмы обработки данных: учеб. пособие / Л.А. Павлов, Н.В. Первова. – 2-е изд. – Чебоксары: Изд-во Чуваш. ун-та, 2018. – 254 с.
4. Павлов, Л.А. Структуры и алгоритмы обработки данных: учебник / Л.А. Павлов, Н.В. Первова. – 3-е изд. – СПб.: Лань, 2021. – 256 с.
5. Свердлов, С.З. Языки программирования и методы трансляции: Учебное пособие / С.З. Свердлов. – СПб.: Питер, 2007. – 638 с.
6. Свердлов, С.З. Языки программирования и методы трансляции: учеб. пособие / С.З. Свердлов. – 2-е изд. – СПб.: Лань, 2019. – 564 с.
7. Теория языков программирования и методы трансляции: Метод. указания к выполнению расчетно-графических работ / Сост. Л.А. Павлов. – Чебоксары: Изд-во Чуваш. ун-та, 2015. – 48 с.